

SQL JOINS

Guía completa para principiantes + ejercicio resuelto

PARTE 1 · Tipos de JOIN explicados

Un **JOIN** es la instrucción SQL que combina filas de dos tablas basándose en una columna relacionada. Antes de escribir código, conviene entender qué pregunta responde cada tipo: la diferencia no es técnica sino conceptual.

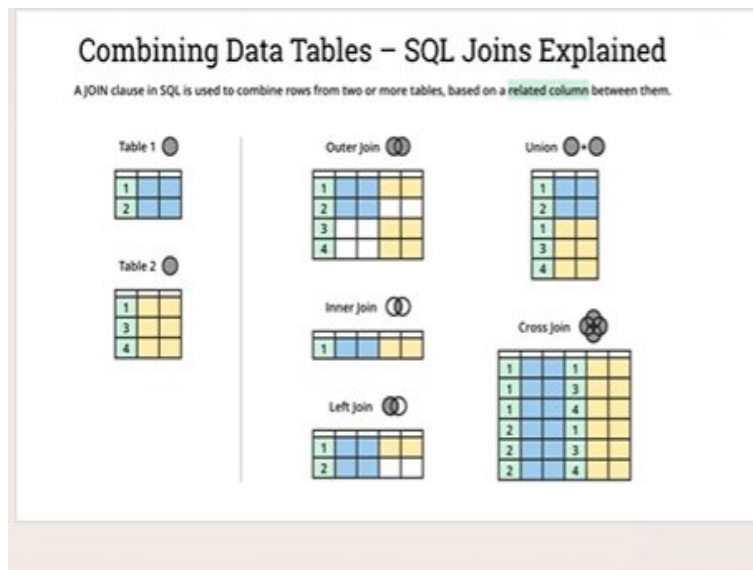


Imagen de referencia: representación visual de los distintos tipos de JOIN en SQL.

INNER JOIN

Solo lo que tienen en común

Pregunta: ¿qué filas existen en ambas tablas a la vez?

Solo entran las filas que tienen pareja exacta en la otra tabla. Si una fila no encuentra correspondencia al otro lado, **desaparece del resultado**. Es el join más restrictivo y el más frecuente en la práctica. Nunca produce valores **null** por la unión en sí: si aparece un null, es porque el dato ya era null en origen.

Regla mental: imagina dos círculos de Venn. El INNER JOIN es solo la zona central donde se solapan, sin nada de los extremos.

LEFT JOIN

Todo lo de la izquierda, lo que pueda de la derecha

Pregunta: ¿qué tienen todos mis registros de la izquierda, aunque no tengan pareja?

La tabla **izquierda manda**: todas sus filas aparecen en el resultado sí o sí. Si una fila de la izquierda tiene pareja en la derecha, se combinan. Si no la tiene, la fila aparece igualmente pero con **null** en todas las columnas de la derecha.

Caso de uso típico: 'dame todos los pedidos, y si tienen cliente asociado dime cuál; si no, ponme null'. Es el join más utilizado en aplicaciones reales.

RIGHT JOIN

Todo lo de la derecha, lo que pueda de la izquierda

Pregunta: ¿qué tienen todos mis registros de la derecha, aunque no tengan pareja?

Es el espejo exacto del LEFT JOIN: ahora son las filas de la **derecha** las que siempre aparecen. Si no tienen pareja en la izquierda, las columnas de la izquierda reciben **null**. En la práctica se usa menos porque puedes conseguir el mismo resultado cambiando el orden de las tablas y usando LEFT JOIN.

FULL OUTER JOIN

Todo de ambos lados, sin excluir nada

Pregunta: ¿qué existe en cualquiera de las dos tablas?

El más inclusivo de todos: combina LEFT y RIGHT a la vez. Conserva **todas las filas de la izquierda Y todas las de la derecha**. Donde no hay pareja, se rellena con **null** en las columnas del lado que falta.

Útil para auditorías: 'muéstrame todo lo que tengo en cualquiera de los dos sistemas y señala qué no tiene equivalente en el otro'.

UNION

Filas apiladas, no columnas combinadas

Atención: esto no es un JOIN. Es un mecanismo distinto.

Un UNION **no mezcla columnas**, sino que **apila filas**: toma los resultados de dos SELECT y los pone uno debajo del otro, como si pegaras dos hojas de papel. Ambas consultas deben tener el mismo número de columnas y tipos compatibles. **UNION** elimina duplicados; **UNION ALL** los conserva (más rápido).

CROSS JOIN

Producto cartesiano: todas las combinaciones posibles

Pregunta: ¿cuántas combinaciones distintas existen entre cada fila de T1 y T2?

No usa ninguna condición de unión. Combina **cada fila de la izquierda con cada fila de la derecha**, sin filtrar nada. Si T1 tiene 2 filas y T2 tiene 3, el resultado tiene $2 \times 3 = 6$ filas. Caso de uso clásico: tabla de tallas (S, M, L) $\times$ tabla de colores (rojo, azul) = catálogo completo de combinaciones.

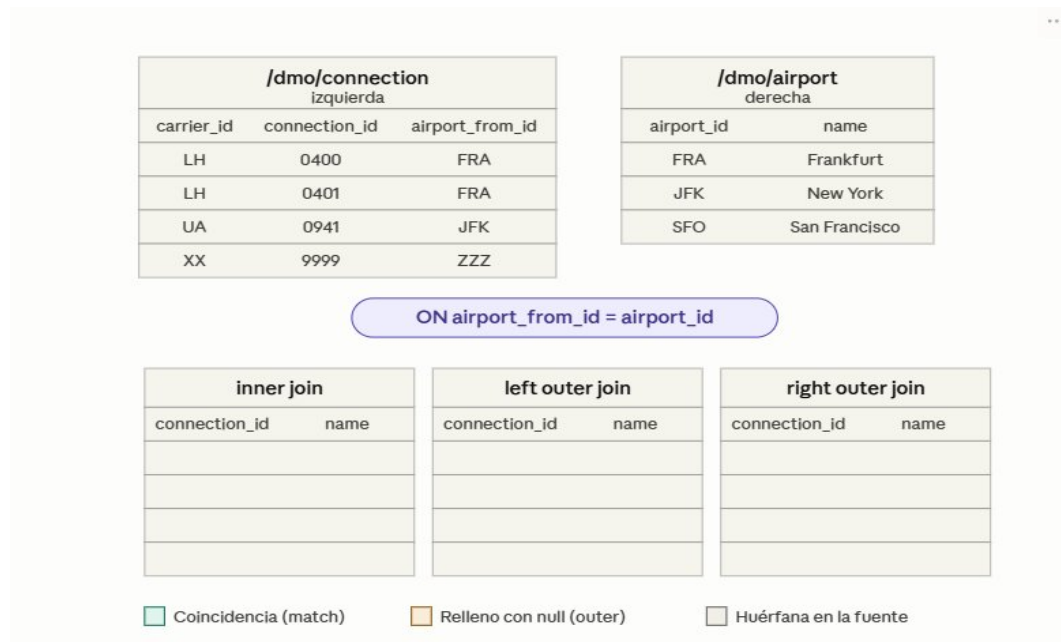
Precaución: con tablas grandes puede ser peligroso. $1.000 \times 1.000 = 1.000.000$ filas.

Regla mental para recordarlos: pregúntate '¿qué tabla quiero que mande en el resultado?'. Si quieres que manden **ambas** → FULL OUTER. Si manda solo la **izquierda** → LEFT. Si manda solo la **derecha** → RIGHT. Si solo quieres **lo que casa** → INNER. Si quieres **todas las combinaciones posibles sin condición** → CROSS.

PARTE 2 · Ejercicio resuelto

Datos del ejercicio

Se trabaja con dos tablas del modelo de vuelos. Cada conexión sale de un aeropuerto guardado en **airport_from_id**, que se corresponde con el **airport_id** de la tabla de aeropuertos.



/dmo/connection izquierda		
carrier_id	connection_id	airport_from_id
LH	0400	FRA
LH	0401	FRA
UA	0941	JFK
XX	9999	ZZZ

/dmo/airport derecha	
airport_id	name
FRA	Frankfurt
JFK	New York
SFO	San Francisco

ON airport_from_id = airport_id

inner join	
connection_id	name

left outer join	
connection_id	name

right outer join	
connection_id	name

☒ Coincidencia (match) ☐ Relleno con null (outer) ☐ Huérfana en la fuente

Enunciado original del ejercicio con las tablas /dmo/connection y /dmo/airport.

Condición de unión: ON connection.airport_from_id = airport.airport_id

Paso 1 y 2 — Emparejamiento previo

Antes de construir cada join, identificamos qué filas tienen pareja y cuáles no:

carrier_id	connection_id	airport_from_id	→ airport_id	name
LH	0400	FRA	FRA ✓	Frankfurt
LH	0401	FRA	FRA ✓	Frankfurt
UA	0941	JFK	JFK ✓	New York
XX	9999	ZZZ	– sin pareja	–
–	–	–	SFO ■	San Francisco

Coincidencia (match)

Relleno con null (outer)

Huérfana en la fuente

Paso 3 — Las tres tablas de resultado

INNER JOIN

connection_id
0400
0401
0941

→ 3 filas en total

LEFT OUTER JOIN

connection_id
0400
0401
0941
9999

→ 4 filas en total

RIGHT OUTER JOIN

connection_id
0400
0401
0941
null

→ 4 filas en total

Coincidencia (match)

Relleno con null (outer)

Huérfana en la fuente

Respuestas a las preguntas del ejercicio

P1. ¿Cuántas filas tiene cada uno de los tres resultados?

El **INNER JOIN** produce **3 filas** (solo las coincidencias). El **LEFT OUTER JOIN** produce **4 filas** (las 3 coincidencias más la conexión 9999 sin aeropuerto). El **RIGHT OUTER JOIN** produce **4 filas** (las 3 coincidencias más San Francisco sin ninguna conexión).

P2. ¿Por qué Frankfurt aparece más de una vez? ¿Qué pasa cuando un valor de la izquierda tiene varias coincidencias?

Porque **dos conexiones distintas** (0400 y 0401) tienen el mismo **airport_from_id = FRA**. El join genera una fila de resultado por cada pareja que encaja: como FRA existe en la tabla de aeropuertos, se combina con cada una de las dos conexiones que salen de allí. Esto revela la relación N→1: *muchas conexiones pueden salir del mismo aeropuerto*. Es comportamiento normal y esperado, no un error.

P3. ¿En qué join aparece la conexión 9999, y por qué con name = null?

Solo en el **LEFT OUTER JOIN**. Su regla es 'conservar todas las filas de la tabla izquierda, tengan pareja o no'. La conexión XX-9999 tiene **airport_from_id = ZZZ**, que no existe en la tabla de aeropuertos. Como no hay pareja, el motor no puede recuperar ningún name y rellena ese campo con **null**. La fila aparece igualmente porque la tabla izquierda manda.

P4. ¿En qué join aparece San Francisco, y por qué con connection_id = null?

Solo en el **RIGHT OUTER JOIN**. Su regla es 'conservar todos los registros de la tabla derecha, tengan pareja o no'. El aeropuerto SFO existe en /dmo/airport, pero ninguna conexión de /dmo/connection tiene **airport_from_id = SFO**. Como no hay pareja, no existe ningún connection_id que devolver → **null**. SFO es un aeropuerto huérfano de la derecha.

P5. ¿Qué join no muestra ninguna fila con null? ¿Por qué?

El **INNER JOIN**. Su regla es la más estricta: solo pasan al resultado las filas que tienen pareja exacta en *ambos* lados. Si no hay coincidencia, la fila desaparece completamente, nunca se rellena con null. Por eso es el join más 'limpio', pero también el que más datos puede descartar si hay registros sin correspondencia.